Práctica 1. Organización de ficheros

Autor: Lucas Hidalgo Herrera

Grado: Doble Grado en Ingeniería Informática y Matemáticas

Asignatura: Administración de Bases de Datos

Profesor: Ignacio José Blanco Medina

Fecha: 22 de marzo de 2026

Índice General

1 Introducción **1**

2 Estructuras soporte **1**

3 TDA fileRecord **2**
3.1 Estructura en memoria 2
3.2 Operaciones públicas 3
3.3 Operaciones Privadas 4

4 TDA fileBlock **5**
4.1 Estructura en memoria 5
4.2 Operaciones públicas 5
4.3 Operaciones privadas 8

5 TDA fileHeader **9**

6 TDA fileASF **10**
6.1 Estructura en memoria 10
6.2 Operaciones privadas 10

1 Introducción

A lo largo de este documento se va a realizar la práctica 1 de la asignatura de *Administración de Bases de Datos*. Dicha práctica consiste en documentar, diseñar y especificar algunos tipos de datos abstractos para el trabajo con ficheros de carácter secuencial físico.

Estos tipos de datos abstractos están basados en la estructura en memoria de los ficheros del sistema operativo; que como vimos en clase, un fichero no es más que una secuencia de bloques junto a una cabecera, donde cada bloque es una secuencia de registros precedidos globalmente por una cabecera.

De cada uno de ellos, trataremos la *estructura en memoria*, las *operaciones públicas* y las *operaciones privadas* [1]. Por tanto, debemos especificar y diseñar los tipos de datos abstractos **TDA fileASF**, **TDA fileHeader**, **TDA blockFile** y **TDA fileRecord**.

2 Estructuras soporte

Con vistas a trabajar con mayor facilidad vamos a definir una serie de estructuras que trataremos como una sola. Esto es a causa de que son estructuras unificadas para todos los objetos de un mismo tipo de dato abstracto.

Estas estructuras serán dos:

1. *unifiedRecord*: esta estrucutra refleja el conjunto de atributos del tipo de dato abstracto *TDA fileRecord* que están unificados para todos los objetos del mismo .

   Esta estructura está compuesta por los atributos *headerRecordSize*, *numAttr*, *tipoCampos* y *camposSize*; es decir, es la siguiente

```
// Representación en memoria del registro
private:
estructura unifiedRecord {
    entero headerRecordSize;
    entero numAttr;
    secuencia<entero> tipoCampos;
    secuencia<entero> camposSize;
}
```

Listing 1: Estructura *unifiedRecord*

2. *unifiedBlock*: esta estrucutra refleja el conjunto de atributos del tipo de dato abstracto *TDA fileBlock* que están unificados para todos los objetos del mismo .

   Esta estructura está compuesta por los atributos *headerBlockSize* y *blockSize*; es decir, es la siguiente

[1] Todo ello en pseudocódigo

```
// Representación en memoria del registro
private:
estructura unifiedBlock {
    entero headerBlockSize;
    entero blockSize;
}
```

Listing 2: Estructura *unifiedRecord*

Cabe destacar que estas estructuras no son estrictamente necesarias, simplemente se han creado para facilidad de comprensión. Entenderemos que los operadores de igualdad hacen copia parámetro a parámetro; así como que su creación se hace atributo por atributo, siempre que la visibilidad del objeto en el que se encuentra lo permita.

Además, debo comentar que los atributos no son constantes puesto que como estructura no es necesario; sin embargo, una vez formen parte de uno de los tipos de datos abstractos que conforman el fichero sí pasarán a ser constante.

Estas estructuras ayudarán a comprender que, en la situación en la que estamos (bloques homogéneos con registros de longitud fija), esta estructura se impone al principio de forma conjunta y no se puede manipular.

3 TDA fileRecord

Comenzamos por el más interno, lo que ayudará a aprender de forma constructiva el objetivo de la práctica.

3.1 Estructura en memoria

Siguiendo el guión de la práctica, un registro es una secuencia de bytes que almacena dos aspectos, el primero de ellos es la estructura propia del registro y el segundo son los datos siguiendo la estructura que se ha impuesto en el primer dato.

```
// Representación en memoria del registro
private:
estructura fileRecord {
    const unifiedRecord structRecord;
    secuencia<bytes> content;
}
```

Listing 3: Especificación de la estructura del registro en memoria

Como vemos, el atributo *structRecord* que almacena toda la estructura del registro y que es idéntica en todos los de su mismo tipo, es constante. Esto pone de manifiesto que se trabajará como un todo y que no podrá modificarse una vez creado el objeto.

En adición, es conveniente recordar que esta estructura vendrá reflejada en la cabecera del archivo y deberán heredarla del mismo al ser creados para mantener la estructura homogénea de los bloques con registros de longitud fija.

3.2 Operaciones públicas

Puesto que uno de los requisitos que se nos impone en la práctica es que el contenido de los registros pueda ser obtenido por un objeto de tipo *TDA fileASF*, debemos imponer algunas de las funciones de tipo *get* y tipo *set*.

```
// Getter y Setters
public:
/**
* Getter para obtener la estructura del registro
*
* @ret: copia de la estructura del registro
**/
unifiedRecord getStructRecord() const;

/**
* Getter para obtener el contenido de todo el registro
*
* @ret: copia del contenido del registro
**/
secuencia<bytes> getContent() const;

/**
* Setter para cambiar el contenido de todo el registro
*
* @param newContent: secuencia de bytes que alberga el nuevo
    contenido del registro
*
* @pre: content.size()==newContent.size
* @post: No se cambia el tamaño del registro, pero sí su contenido
**/
void setContent(secuencia<bytes> newContent);
```

Listing 4: Operaciones públicas del registro

Con respecto al único *setter* que he creado, he supuesto que los datos del nuevo contenido tienen el mismo tipo; aún así, si no lo fuera, considero que la base de datos proporcionaría el error.

Entonces, en caso de que fuera necesario, se podría imponer la condición *newContent[i].type==tipoCampos[i]*, pero no soy muy partidario de este tipo de prácticas.

Para finalizar este aspecto, siguiendo la estructura que se nos propone en el ejercicio, no debemos realizar ningún otro *Setter*, ya que tratamos con bloques de estructura **homogénea** y con registros de longitud fija, es decir, cambiar *camposSize*, *headerSize*, *tipoCampos* o *numAttr* está en contra de esta estructura.

Como aclaración, el motivo por el cual todos los *getter* devuelven una copia es para evitar posibles modificaciones por referencia dependiendo del lenguaje de programación que se use; verificando así que no se pueden modificar datos que no se deban modificar.

Por último, debemos crear un constructor y un destructor de este tipo de dato.

```
// Constructor y destructor del registro
public:
```

```
/**
* Crea una instancia del tipo fileRecord con los datos pasados como
    argumentos
* @param structRecord: estructura del registro a crear
* @param cont: contenido de cada uno de los campos del registro
*
* @pre: structRecord.hSize>0, structRecord.nAttr>0
* @post: Se crea una instancia de tipo fileRecord
**/
fileRecord (unifiedRecord structRecord, secuencia<bytes> content);

/**
* Constructor de copia
* @param otherFileRecord: registro a copiar
*
* @pre: otherFileRecord != null
* @post: Se crea una instancia de tipo fileRecord idéntica a
    otherFileRecord
**/
fileRecord (fileHeader otherFileRecord);

/**
* Destruye una instancia de fileRecord
*
* @post: Se crea una instancia de tipo fileRecord
**/
~fileRecord()
```

Listing 5: Constructor y destructor del registro

3.3 Operaciones Privadas

Como cabia de esperar, considero que este tipo de dato no admite operaciones privadas debido a que no dispone de una funcionalidad activa. Es decir, es claro que la principal funcionalidad de este tipo de dato es portar información de memoria a disco y de disco a memoria.

Por tanto, desde mi punto de vista, las operaciones privadas exigidas para el manejo del disco se delegan al tipo de dato abstacto *TDA fileASF*.

No obstante, podríamos pensar en una operación privada que realice una de las comprobaciones más importantes, la verificación de tamaños en la creación de una instancia de este tipo de dato; tendría la siguiente especificación:

```
// Operaciones privadas
private:
/**
* Comprueba las restricciones de que los tamaños y cantidades sean
    positivos así como que el tamaño del contenido sea igual a la
    suma de los tamaños especificados en camposSize al estar en un
    ASF. Además verifica que haya tantos tipos de campos como numero
     de atributos especificados. En definitiva, realiza las
    comprobaciones necesarias para no crear un objeto incorrecto.
*
* @param structRecord: estructura del registro
```

```
* @param cont: contenido de cada uno de los campos del registro
*
* @ret: true -> tamaños positivos y dentro de las restricciones de tipos
**/
boolean checkSizes (unifiedRecord structRecord, secuencia<bytes> content);
```

Listing 6: Operaciones privadas del registro

4 TDA fileBlock

Está claro que un fichero es un conjunto de registros. Sin embargo, la memoria no trabaja con registros como tal, sino que trabaja con bloques de memoria. Por tanto, puesto que buscamos construir un tipo de dato abstracto que modele un fichero, trataremos un bloque como una agrupación de registros [2].

Con objetivo de modelar con mayor facilidad, y siguiendo las instrucciones del guión del trabajo, vamos a crear la siguiente estructura que modela cómo se guarda cada registro en el bloque.

```
// Representación en memoria del registro
private:
secuencia<bytes> recordBlock {
    entero idRecord;
    boolean stateRecord;
    secuencia<bytes> contentRecord;
}
```

Listing 7: Especificación de la estructura de un registro en un bloque

4.1 Estructura en memoria

```
// Representación en memoria del registro
private:
estructura fileBlock {
    entero idBlock;
    secuencia<recordBlock> records;
}
```

Listing 8: Especificación de la estructura del bloque del fichero en memoria

4.2 Operaciones públicas

```
// Constructor y destructor de bloque
public:
/**
* Crea una instancia del tipo fileBlock con los datos pasados como argumentos
* @param idBlock: identificador de Bloque
```

[2] Evidentemente, no todos los bloques de memoria son agrupaciones de registros, depende de la finalidad del programa que lo use, pueden ser fragmentos de código a ejercutar...

```
* @param records: registros del bloque
*
* @pre: idBlock>0, records.idRecord[i]>0
* @post: Se crea una instancia de tipo fileBlock
**/
fileBlock (entero idBlock, secuencia<recordBlock> records);

/**
* Crea una instancia de tipo fileBlock vacía, solo con el idBlock
* @param idBlock: identificador del bloque
*
* @pre: numRecords>=0, idBlock>0
* @post: Se crea una instancia de tipo fileBlock vacía
**/
fileBlock (entero idBlock);

/**
* Destruye una instancia de fileRecord
*
* @post: Se crea una instancia de tipo fileRecord
**/
~fileBlock()
```

Listing 9: Constructor y destructor de bloque

```
// Constructor y destructor de bloque
public:
/**
* Añade un registro al bloque; en caso de que el bloque esté completo devolverá -1
*
* @param record: registro a añadir
* @param structRecordFile: estructura de los registros del fichero
*
* @pre record.structRecord = structRecordFile
* @post: se añade el registro al bloque si es posible por tamaño, el regisro dispondrá de su identificador de registro y de estado a true.
*
* @ret: entero que determina el estado de la operación:
*       - (-1) el bloque está completo y se debería crear otro bloque
*       - (0) no se ha podido realizar la operación, pero el bloque dispone de espacio.
*       - (1) la operación se ha realizado correctamente
*/

entero addRecord(fileRecord record, unifiedRecord structRecordFile);


/**
* Borra un registro del bloque
*
* @param idRecord: identificador del registro a borrar
*
```

```
* @pre: idRecord > 0
* @post: si se encuentra el registro en el bloque, éste se borra de forma lógica
* @ret: entero que determina el estado de la operación:
*       - (-1) no se ha encontrado el registro
*       - (0) el registro ya está borrado
*       - (1) la operación se ha realizado correctamente
*/
entero deleteRecord(entero idRecord);

/**
* Borra un registro del bloque por clave
*
* @param clave: clave del registro a borrar
* @param structFileRecord: estructura de los registros del fichero
*
* @pre: clave.type==structFileRecord.tipoCampos[clave]
* @post: si se encuentra el registro en el bloque, éste se borra de forma lógica
* @ret: entero que determina el estado de la operación:
*       - (-1) no se ha encontrado el registro
*       - (0) el registro ya está borrado
*       - (1) la operación se ha realizado correctamente
*/
entero deleteRecord(secuencia<bytes> clave, unifiedrecord structFileRecord);

/**
* Borra un registro del bloque que sea idéntico al pasado como argumento
*
* @param record: registro a borrar
* @param structFileRecord: estructura de los registros del fichero
*
* @pre: record.unifiedRecord==structFileRecord
* @post: si se encuentra el registro en el bloque, éste se borra de forma lógica
* @ret: entero que determina el estado de la operación:
*       - (-1) no se ha encontrado el registro
*       - (0) el registro ya está borrado
*       - (1) la operación se ha realizado correctamente
*/
entero deleteRecord(fileRecord record, unifiedRecord structFileRecord);

/**
* Modifica el contenido de un registro
*
* @param idRecord: identificador del registro a modificar
* @param content: contenido del nuevo registro
* @param structFileRecord: estructura de los registros del fichero
*
* @pre: idRecord > 0, la estructura del contenido es la adecuada
* @post: si se encuentra el registro en el bloque, se borra de forma lógica y se añade uno nuevo con el contenido correcto
* @ret: bloque en el que se encuentra el registro modificado
```

```
*/
fileBlock modifyRecord(entero idRecord, secuencia<bytes> content,
    unifiedRecord structFileRecord);

/**
* Modifica el contenido de un registro idéntico al pasado como
    argumento
*
* @param record: registro a modificar
* @param content: contenido del nuevo registro
* @param structFileRecord: estructura de los registros del fichero
*
* @pre: la estructura del contenido es la adecuada
* @post: si se encuentra el registro en el bloque, se borra de
    forma lógica y se añade uno nuevo con el contenido correcto
* @ret: bloque en el que se encuentra el registro modificado
*/
fileBlock modifyRecord(fileRecord record, secuencia<bytes> content,
     unifiedRecord structFileRecord);

/**
* Modifica el contenido de un registro buscado por clave
*
* @param clave: clave del registro a borrar
* @param content: contenido del nuevo registro
* @param structFileRecord: estructura de los registros del fichero
*
* @pre: la clave es la adecuada, la estructura del contenido es la
    adecuada
* @post: si se encuentra el registro en el bloque, se borra de
    forma lógica y se añade uno nuevo con el contenido correcto
* @ret: bloque en el que se encuentra el registro modificado
*/
fileBlock modifyRecord(secuencia<bytes> clave, secuencia<bytes>
    content, unifiedRecord structFileRecord);
```

Listing 10: Operaciones públicas de bloque

4.3 Operaciones privadas

```
// operaciones privadas de bloque
private
/**
* Carga el bloque de disco a memoria y coloca un puntero en el
    primer registro del bloque.
*
* @param idBlock: identificador del bloque dentro del fichero
*
* @pos: no se modifica nada, simplemente abre el contenido del
    bloque en modo "lectura".
* @ret: entero que determina el estado de la operación:
*       - (-1) no se ha encontrado el bloque en memoria
*       - (0) el bloque ya está abierto
*       - (1) la operación se ha realizado correctamente
*/
```

```
entero bOpen(idBlock) const;

/**
* Escribe el bloque de memoria a disco y cierra el bloque.
*
* @post: el contenido del bloque queda grabado en disco.
* @ret: entero que determina el estado de la operación:
*       - (-1) no se ha podido completar la operación
*       - (0) el bloque ya está cerrado
*       - (1) la operación se ha realizado correctamente
*/
entero bClose() const;

/**
* Busca un registro por identificador de registro
*
* @param idRecord: identificador de registro
*
* @post: simplemente busca hacia delante, nunca retrocede.
* @ret: Si idRecord es mayor que el identificador de registro del
    registro en el que se encuentra el iterador o no encuentra el
    registro, devuelve un registro con identificador -1. En caso
    contrario, devuelve el registro encontrado.
*/
fileRecord find(entero idRecord) const;

/**
* Busca un registro por clave de registro
*
* @param clave: clave del registro
*
* @post: simplemente busca hacia delante, nunca retrocede.
* @ret: Si no encuentra el registro, devuelve un registro con
    identificador -1. En caso contrario, devuelve el registro
    encontrado.
*/
fileRecord find(secuencia<bytes> clave) const;
```

Listing 11: Operaciones privadas de bloque

5 TDA fileHeader

En principio, este tipo de dato no es necesario y, de hecho, sería desconocido para el usuario. El motivo de esto es el siguiente; cuando trabajamos con un fichero, la cabecera dispone de todos los datos del fichero inherentes al propio fichero y que son exclusivos de él. Por tanto, no tiene sentido crear un tipo de dato abstracto para la cabecera.

Aún así, voy a crear una estructura de datos que me ayudará a trabajar con ella en el modelaje del *TDA fileASF*; la llamaré *fileHeader* y su gestión se realizará en el tipo de dato que la contiene.

```
// Representación en memoria de una cabecera de fichero
estructura fileHeader {
```

```
    const entero tamHeader;
    entero numBlocks;
    entero numRecords;
    const unifiedRecord structRecord;
    const unifiedBlock structBlock;
}
```

Listing 12: Especificación de la estructura dela cabecera del fichero en memoria

Como podemos observar en el código superior, la estructura de los registros y de los bloques de memoria será constante, ya que trabajamos exclusivamente con bloques homogéneos cuyos registros disponen de una longitud fija.

Podemos llegar a preguntarnos el por qué de guardar *structRecord* y *structBlock* aquí, pero esto es totalmente necesario para poder gestionar los registros una vez se encuentren fuera del fichero.

6 TDA fileASF

6.1 Estructura en memoria

La estructura en memoria que se nos plantea en el guión es la siguiente:

```
// Representación en memoria de un fichero ASF
private:
estructura fileASF {
    secuencia<caracter> diskFilename;
    diskFile fileManager;
    fileHeader header;
    fileBlock chargedBlock;
    boolean changedHeader;
    boolean changedContent;
}
```

Listing 13: Especificación de la estructura del fichero en memoria

6.2 Operaciones privadas

```
// Operaciones privadas de fileASF
private:

/**
* Obtiene un manejador de fichero para el fichero pasado como
    argumento
*
* @param path: ruta del archivo que se quiere abrir.
*
* @pre: path debe existir en el sistema, en caso contrario da un
    error
* @post: this.diskFilename = path
*        el archivo no es modificado aunque puede ser modificado
    con posterioridad.
*/
diskFile fOpen(secuencia<caracter> path);
```

```

/**
* Cierra el fichero actual
*
* @post: this.fileManager=nullptr
*
* ret: codigo de operación:
*       (-1) no se ha podido cerrar el fichero o no existe
*       (0) el bloque de memoria o la cabecera están modificados
*       (1) se ha cerrado correctamente
*/
entero fClose();

/**
* Cierra y abre el fichero cargando el primer bloque.
*
* @post: this.diskFilename = path
*        el archivo no es modificado aunque puede ser modificado con posterioridad.
*        fileManager se impone en la primera posición del fichero.
* ret: codigo de operación:
*       (-1) no se ha podido realizar la operación o hay inconsistencias.
*       (0) el bloque de memoria o la cabecera están modificados
*       (1) se ha cerrado y abierto correctamente, fileManager está en posición correcta.
*/
entero fCloseAndOpen();

/**
* Obtiene un iterador sobre el bloque del fichero con identificador pasado como argumento
*
* @param idBlock: identificador del bloque en el fichero
*
* @pre: 0<idBlock<=header.numBlocks
* @post: el bloque puede ser modificado
*
* ret: iterador apuntando a la primera posición de memoria correspondiente al bloque.
*/
fileASF::iterator findBlock( entero idBlock );

/**
* Obtiene un iterador sobre los registros del bloque que contiene al registro con clave pasada como argumento.
*
* @param clave: clave del registro que se busca
*
* @pre: clave tiene una estructura adecuada, siguiendo la que hay en header.structRecord.
* @pos: el registro podrá ser modificado.
*       Cierra y abre el fichero para comenzar a buscar desde el principio.
*
* ret: iterador sobre los registros del bloque apuntando al
```

```
    registro encontrado.
*/
fileBlock::iterator findRecord(secuencia<bytes> clave);

/**
* Obtiene un iterador sobre los registros del bloque que contiene
    al registro pasado como argumento.
*
* @param record: registro que se busca.
*
* @pre: record tiene una estructura adecuada, siguiendo la que hay
    en header.structRecord.
* @pos: el registro podrá ser modificado.
*       Cierra y abre el fichero para comenzar a buscar desde el
    principio.
*
* ret: iterador sobre los registros del bloque apuntando al
    registro encontrado.
*/
fileBlock::iterator findRecord(fileRecord record);

/**
* Escribe el contenido del bloque en memoria.
*
* @pre: changedContent==true;
* @pos: el bloque queda almacenado en memoria poniendo el flag
    changedContent a false.
*
* @ret: codigo de estado de la operación
*       (-1) no se ha podido escribir el bloque en memoria o hay
    alguna inconsistencia con la cabecera.
*       (0) el bloque no ha sido modificado.
*       (1) operación realizada correctamente.
*/
entero writeBlockOnDisk() const*;

/**
* Escribe el contenido de la cabecera en memoria.
*
* @pre: changedHeader==true;
* @pos: la cabecera del fichero queda almacenada en memoria
    poniendo el flag changedHeader a false.
*
* @ret: codigo de estado de la operación
*       (-1) no se ha podido escribir la cabecera en memoria o hay
    alguna inconsistencia con el contenido.
*       (0) la cabecera no ha sido modificada.
*       (1) operación realizada correctamente.
*/
entero writeHeaderOnDisk() const*;

/**
* Escribe todo el objeto en memoria.
*
* @pre: changedHeader==true and changedContent==true;
* @pos: fileheader y chargedBlock quedan almacenados en memoria con
```

```
    changedHeader=false=changedContent.
*
* @ret: codigo de estado de la operación
*       (-1) no se ha podido realizar la operación o hay alguna inconsistencia.
*       (0) ningún objeto ha sido modificado
*       (1) operación realizada correctamente.
*/
entero writeOnDisk() const*;

/**
* Añade un registro a la cabecera
*
* @pos: changedHeader=true y header.numRecords+=1.
* @ret: codigo de estado de la operación
*       (-1) no se ha podido realizar la operación.
*       (1) operación realizada correctamente.
*/
entero addRecordOnHeader();

/**
* Sustrae un registro a la cabecera
*
* @pos: changedHeader=true y header.numRecords-=1.
* @ret: codigo de estado de la operación
*       (-1) no se ha podido realizar la operación.
*       (1) operación realizada correctamente.
*/
entero deleteRecordOnHeader();

/**
* Añade un bloque a la cabecera
*
* @pos: changedContent=true y header.numBlocks+=1.
* @ret: codigo de estado de la operación
*       (-1) no se ha podido realizar la operación.
*       (1) operación realizada correctamente.
*/
entero addBlockOnHeader();

/**
* Sustrae un bloque a la cabecera
*
* @pos: changedContent=true y header.numBlocks-=1.
* @ret: codigo de estado de la operación
*       (-1) no se ha podido realizar la operación.
*       (1) operación realizada correctamente.
*/
entero deleteBlockOnHeader();


/**
* Obtiene una lista con todos los registros del fichero que no están modificados
*
* @pos: chargedBlock podría no ser el cargado inicialmente.
```

```
* @ret: secuencia de fileRecord con los registros no borrados del
    fichero de partida
*/
secuencia<fileRecord> getNonDeleted() const

/**
* Copia la lista de registros pasada como argumento al final del
    fichero pasado como argumento.
*
* @param recordSequence: secuencia de registros a copiar
* @param newFile: fichero en el que copiarlos
*
* @pos: newFile cambia su contenido.
*/
void copyOnFile(secuencia<fileRecord> recordSequence, fileASF
    newFile) const*;
```

Listing 14: Operaciones privadas del fichero en memoria

```
// Iterador sobre bloques
private:
class iterator{
    private:

    pointer pt;

    public:
    iterator(pointer p);

    iterator& operator++();

    fileBlock& operator*();
}
```

Listing 15: Iterador sobre los bloques del fichero

6.3 Operaciones públicas

```
// Constructor y destructor
public:

fileASF(diskFilename path);

fileASF(fileASF newFile);

~fileASF();

}
```

Listing 16: Constructor y destructor del fichero

```
// Constructor y destructor
public:

fileRecord getRecord(secuencia<caracter> clave) const;

```

```
fileASF reorder() const;

entero insertRecord(fileRecord record);

entero insertRecordSequence(sequence<fileRecord> records);

entero deleteRecord(secuencia<caracter> clave);

entero deleteRecordSequence(secuencia<secuencia<caracter>> claves);

entero modifyRecord(fileRecord record);

entero modifyRecordSequence(secuencia<fileRecord> records);

}
```

Listing 17: Operaciones públicas del fichero